

Group Project

Evaluating and Improving Large Language Model Systems

PolyU 2025/26 Semester 3

Questions: ask in the [shared FAQ sheet](#), or email the TA (sheildon.shen@connect.polyu.hk) if necessary

Team registration: fill in the [team-formation sheet](#)

1 Task Description

Large language models (LLMs) are now widely used for reasoning, writing, dialogue, summarization, and information access. Their performance, however, varies across tasks, languages, prompts, model versions, and system designs. This group project asks you to study LLMs through hands-on experiments rather than subjective impressions.

Each team chooses one of the two tracks. **Track A** focuses on the comparative evaluation of multiple LLMs. **Track B** focuses on improving an LLM-based system using a small, well-justified workflow technique and proving whether the improvement works. The two tracks differ in form, but share the same core expectation: define a clear task, design suitable metrics, collect complete evidence, and explain the results honestly.

Compute and model access (no GPU required). Everything in this project runs on free resources. You will use accessible LLMs through their APIs or interfaces; there is no fixed model list, and *finding and choosing suitable (free-tier) models is part of your work*. Always record the exact model name, version (if available), access route, and access date. Keep datasets small and cache responses so that you stay within free quotas.

2 Project Tracks

2.1 Track A: Comparative Evaluation of LLMs

The goal of Track A is to conduct a systematic evaluation of popular LLMs (e.g. GPT, Claude, Gemini, DeepSeek, Qwen) across multiple capability directions. Most of your effort should go into metric design, model comparison, and error analysis, not merely collecting model outputs.

Evaluation directions

The following six directions are provided as fixed starting points:

1. Semantic Understanding
2. Abstractive Summarization
3. Conversations and Dialogue
4. Reasoning
5. Knowledge and Factuality
6. Character Simulation

In addition to the fixed directions, each team must include at least one self-proposed evaluation direction that is not identical to any of the six.

Workload and team-size rule for Track A

Team size	Required directions	Min. self-proposed	Prompt workload
2 members	4 directions in total	At least 1	at least 30 prompts per direction
3 members	6 directions in total	At least 1	at least 30 prompts per direction
4 members	8 directions in total	At least 2	at least 30 prompts per direction

Each member should lead two evaluation directions. Duplicated directions within a team are not allowed unless approved. Every prompt must be tested on the same set of LLMs.

Track A: required steps

1. **Select evaluation directions:** each member leads two; at least one group-level direction is self-proposed and different from the six fixed ones.
2. **Select LLMs:** evaluate at least three accessible LLMs; the same LLMs must be used by all members. Record exact name, version, access route, and access date.
3. **Design evaluation metrics:** for each direction, design metrics that reflect the capability being tested. A good metric set usually has several dimensions and clear scoring rules.
4. **Design prompts:** create at least 30 prompts per direction. Prompts should be diverse enough to reveal model differences, not repetitive paraphrases.
5. **Collect responses and scores:** run all selected LLMs on the same prompts; document responses, scores, and comments in the project records.
6. **Analyze performance:** compare strengths and weaknesses using quantitative scores and qualitative failure cases; explain why the differences matter.

A worked example of one direction (Reasoning), with example metrics, weights, and prompts, is given in [Appendix A](#).

2.2 Track B: Design and Improve a Small LLM Workflow

In Track B, you first pick a task: something that a single LLM call does *not* handle reliably. You then design a small workflow around the model that does the task better, and prove, with your own evaluation, that the workflow beats the single-call baseline.

A workflow is the process around the model; the baseline is the simplest possible workflow, a single call. You improve the workflow by adding a "harness": a control step that steers the model toward a more reliable answer, for example, retrieving facts before answering, or checking and revising the answer. You may add one harness or stack several. The interesting question, and the heart of your evaluation, is whether each harness you add actually helps.

The word *harness* comes from horse tack: it is the gear a rider adds to steer and control a powerful horse. An unharnessed horse runs where it likes; with a harness you can direct it. A single LLM call is the unharnessed model; the steps you add are the harness that steers it.

Example: RAG, the classic harness

The clearest example of adding a harness is retrieval (RAG). Before the model answers, your program searches a collection of documents for the passages most relevant to the question and pastes them into the prompt, so the model answers *from those passages* instead of from memory. This reduces made-up answers and lets you check whether the answer is grounded in the retrieved text. RAG fits tasks where the answer should come from a specific body of text, a product FAQ, a policy document, a set of lecture notes.

Appendix B walks through a full RAG example end to end, and shows other harnesses you can add (a self-check / revise step, a separate verifier) and how to measure whether each one helps. Read it for the details.

You may use RAG, combine harnesses, or design a different one of your own. Keep the workflow small and easy to explain, and stay within the scope of the course, no model fine-tuning, and no complex multi-agent systems. *Minor prompt tweaks (few-shot examples, output-format instructions) may support your workflow, but do not count as the main contribution on their own.*

Workload and team division for Track B

A Track B team works on one single, specific task, chosen together so that it fits the team's professional or disciplinary background (for example, a finance, healthcare, education, law, or engineering setting that your team understands well). The whole project revolves around this one task; do not split into unrelated tasks.

Within that shared task:

- Each member must design and contribute at least one harness, a control step added to the shared workflow (for example, one member adds retrieval, another a self-check step, another a verifier).
- The team must measure each member's harness individually against the single-call baseline (does it help, where, and by how much?).
- The team must then combine the members' harnesses into one final workflow and measure the combined effect, does stacking the steps help further, or do some steps add little once others are present?

Team size	Harnesses (min.)	What must be measured
2 members	at least 2	each harness individually + the final combined workflow, vs. the baseline
3 members	at least 3	each harness individually + the final combined workflow, vs. the baseline
4 members	at least 4	each harness individually + the final combined workflow, vs. the baseline

Track B: required steps

1. **State the need:** choose a concrete task. It does not have to be text-based, for example: answering questions over documents, extracting structured data, classifying or grading inputs, or an agent that plays a simple (e.g. text-based) game. Explain who would need it and what a good output means.
2. **Build a baseline:** a sensible single-LLM-call baseline, the system your improved workflow must beat (or explain why it cannot).
3. **Build the harnesses and the combined workflow:** each member designs at least one harness (a control step such as retrieval, self-check, or a verifier) for the shared task. The team then stacks the members' harnesses into one final workflow. Keep each one simple enough to explain clearly.
4. **Design an evaluation set:** assemble a set of test cases --- each an input paired with a known correct answer, reference, source passage, or a verifiable scoring criterion --- that you run every version on. Collect enough representative cases to support a credible conclusion rather than a few cherry-picked examples: too few and your result could just be noise. *You do not need a huge set; a few dozen varied cases is usually plenty, and larger teams should test more thoroughly.*
5. **Choose metrics:** metrics that fit the need, e.g. task success rate or score, correctness, faithfulness, retrieval hit@k, format accuracy, judge score, human-checked score, cost, latency.
6. **Run comparisons and an ablation:** on the same evaluation set, compare the baseline, each member's harness individually, and the final combined workflow. Include at least one ablation/control (e.g. remove retrieval, remove the verifier, change top-k) to show which harnesses actually contribute.
7. **Analyze results honestly:** if it improves, explain where and why. If it does not, explain the failure clearly. A well-analyzed negative result beats an unsupported positive claim.

Optional extension (if you are interested). If you want to take your workflow further, you can turn it into something closer to a usable tool --- for example, wrap it in a simple web page, a small app, or an interactive notebook, and demo it live. This is entirely optional and just a way to go further if it interests you. The rubric still rewards the evaluation, the evidence, and honest analysis --- *a polished interface is a nice plus, not a substitute for proving that the workflow works.*

3 Common Requirements for Both Tracks

1. **Clear task definition**, define what is being evaluated or improved, and why it matters.
2. **Controlled comparison**, use the same prompts, data, model settings, and conditions whenever comparison is intended.
3. **Appropriate metrics**, metrics must match the task; poorly chosen metrics give weak conclusions even with many experiments.
4. **Complete records**, keep prompts, responses, scores, model names/versions, access dates, and scoring comments.
5. **Quantitative and qualitative analysis**, include representative success cases, failure cases, and explanations.
6. **Limitations and responsible use**, discuss reliability, bias, privacy, cost, latency, and other relevant limitations.
7. **AI-tool disclosure**, you may use AI tools, but disclose which and for what. You must be able to explain your own project in Q&A.

4 Team Formation

Teams should consist of 2 to 4 members. All teams, regardless of size, are evaluated using the same rubric; workload scales mainly through the number of Track A directions or the depth of Track B evaluation.

- Teams with fewer than 2 or more than 4 members require approval with a valid justification.
- Submit your team information using the [team-formation sheet](#) by **the evening of 28 June 2026**: one row per group, with the project name, the chosen track, and each member's student ID and name. **If you have not registered a team by then, you may be assigned to a team at random.**
- Team changes are not allowed after the team-formation deadline unless approved.

5 Submission Guideline

Submission deadline: to be announced on Blackboard. Late submission: 10% deduction per calendar day, up to three days; more than three days late is marked zero unless an approved extension is granted.

Submission package

Submit one compressed folder named `GroupProject_<GroupID>.zip` (or `.rar`). Only one member needs to upload. Double-check that it saved successfully.

File / folder	Required for	Description
records.xlsx / results.csv	Both tracks	Evaluation metrics, prompts/test cases, LLM outputs, scores, comments, summary tables.
report.pdf	Both tracks	Technical report, max 10 pages excluding references and appendices.
slides.pdf	Both tracks	Slides summarizing motivation, methodology, results, conclusions.
code/ + README.md	Mandatory for Track B; optional for Track A	Runnable code/notebook/scripts with a short README on reproducing the experiments.

Project records

Your records are part of the graded submission and must be clear enough for the teaching team to verify what was tested and how scores were assigned.

- **Track A:** first worksheet summarizes metric descriptions; a separate worksheet per direction (owner, prompts, LLM names, versions, responses, scores, comments).
- **Track B:** evaluation set, baseline outputs, each member's harness outputs, the final combined-workflow outputs, ablation/control outputs, scores, judge prompt (if used), human sanity-check notes, and cost/latency observations.

Project report

Begin with all members' names and student IDs. Suggested structure:

1. **Introduction**, task, motivation, and selected track.
2. **Methodology**, Track A categories/models/metrics, or Track B need/baseline/workflow.
3. **Evaluation design**, prompts or test cases, scoring rules, metrics, controls.
4. **Experimental results and analysis**, comparison tables, error analysis, ablation if applicable.
5. **Limitations, responsible-use discussion, and conclusions.**
6. **Acknowledgement:** to acknowledge each team member's individual effort in preparing the final project, this section should clearly describe the specific topic(s) or task(s) each member focused on. You may also include any necessary supplementary information about each member's contribution.

Presentation

Each team gives a live, in-class presentation of **no more than 7 minutes**, followed by Q&A. Cover the same core structure as the report. The team should speak and be ready to answer questions about the project design and evaluation choices.

- **Track A**, emphasize evaluation directions, metrics, model differences, and representative success/failure cases.
- **Track B**, emphasize the need, baseline failure, harness/workflow design, evaluation, ablation, and evidence of improvement (or non-improvement); demo your app or interface live if you built one.

Academic honesty and AI-tool policy

This is group work; collaboration inside the group is expected. Copying another group's design, code, data, prompts, scores, or report is plagiarism and is treated seriously. AI tools may be used, but the intellectual work must be yours. In the report, disclose which AI tools were used and for what purpose. A submission the team cannot explain during Q&A will score poorly even if the written output looks polished.

6 Grading Rubric

Graded out of 100 marks. The same shared rubric applies to both tracks. A single group grade is normally assigned to all members. If the Acknowledgement section (see the report structure above) shows that contributions were clearly uneven, individual marks may be adjusted accordingly; if no breakdown is given, contributions are assumed to be equal.

Criterion	Marks	Shared requirement
Scope and project design	10	Clear goal, suitable scope, justified track choice, feasible division of labour.
Execution and records	20	Experiments completed carefully; required prompts/test cases, outputs, scores, implementation details, and logs are complete and verifiable.
Evaluation metrics and experimental design	25	Metrics relevant to the task, scoring rules clear, comparisons controlled, design supports the project claim.
Results, analysis, and limitations	15	Results summarized quantitatively and analyzed qualitatively; explains model differences or workflow improvement, failure cases, limitations, cost/latency, responsible use.
Quality of project report	10	Clear, well-structured, concise, technically accurate; complete contribution and AI-tool disclosure.
Quality of presentation and Q&A	20	Clearly communicates task, method, evaluation, results, conclusions; team answers questions about design choices and evidence.

Track-specific evidence under the shared rubric

Rubric area	Track A evidence	Track B evidence
Scope and design	Directions follow the team-size rule; ≥ 1 self-proposed; chosen LLMs accessible and used consistently.	One shared task fitting the team's background; each member contributes at least one harness; baseline and workflows clearly defined; techniques appropriate to the task.
Execution and records	Every direction has at least 30 prompts; all tested on ≥ 3 LLMs; responses and scores documented.	Baseline, each member's harness, the combined workflow, and ablation/control run on the same set; code/notebook and logs reproducible.
Metrics and design	Metrics reflect the capabilities and reveal model differences; scoring rules justified and applied consistently.	Evaluation set has ground truth/verifiable criteria; metrics match the task; LLM-as-judge disclosed and sanity-checked.
Results and analysis	Compares model performance across directions; explains strengths, weaknesses, representative failures.	Shows whether the technique beats baseline; explains ablation, failure cases, cost/latency trade-offs.

A technically complex system does not automatically score higher. The project is rewarded for clear design, fair comparison, suitable metrics, complete evidence, and honest analysis.

7 Consultation

A shared [**FAQ sheet**](#) (a Google Sheets document; the link is also posted on Blackboard) is the channel for general questions. Check the answered questions first; if yours is not covered, **post it in the sheet's question area** and the TA will answer there, so the whole class benefits.

Email the TA (sheldon.shen@connect.polyu.hk) or the course instructor only for matters that need direct contact, such as personal or group-specific issues. If you have not received a reply after 48 hours, please resend and copy the course instructor.

Appendix A: Track A Example

Not graded directly: a reference example only. Your team should design your own directions, prompts, metrics, and analysis.

Example direction: Reasoning

Suppose a member selects Reasoning. The goal is not to ask random hard questions, but to test reasoning in a controlled way. Prompts may include logical reasoning, multi-step arithmetic, causal reasoning, counterfactual reasoning, and robustness to irrelevant information.

Metric	Weight	Description	Justification
Final-answer correctness	40%	Whether the final answer is correct under the stated conditions.	The most direct indicator of reasoning success.
Reasoning validity	30%	Whether intermediate steps are logically sound and rely on no unsupported assumptions.	A correct answer with invalid reasoning may not indicate reliable ability.
Instruction following	15%	Whether the model follows requested format, constraints, or explanation length.	Reasoning tasks often include constraints that should be respected.
Robustness	15%	Whether the model stays correct when irrelevant details or paraphrased wording are added.	Reveals whether the model truly understands the problem structure.

Prompt examples

Example 1, logical reasoning: A museum has three rooms: A, B, and C. Each room contains exactly one object: a vase, a painting, or a statue. The vase is not in Room A. The painting is either in Room A or Room C. Room B does not contain the statue. Which object is in each room? Explain your reasoning briefly.

Example 2, robustness to irrelevant information: Ignore the fact that the story takes place on a rainy Tuesday. A train leaves Station X at 9:00 travelling at 60 km/h. Another train leaves Station Y, 180 km away, at 9:30 travelling toward Station X at 90 km/h. At what time do they meet? Show the calculation.

Reporting: apply the same metric weights to every model's response, then summarize average scores, compare model differences, and analyze representative failures (arithmetic errors, unsupported assumptions, missing constraints, overconfident explanations).

Appendix B: Track B Worked Example (with Sheldon, Alice, and Bob)

Read me first: a launchpad for Track B, not a template to copy. Your own Track B project must use a different need, dataset, and evaluation design. *The code below is essentially the whole program: plain Python plus one helper `call_lm()` that sends an HTTP request to your chosen model's API (every provider's documentation shows the few lines needed). No special libraries are required. We have run this exact workflow end to end.*

It is a quiet evening in the lab. The exam is finally behind them, and three chairs have drifted around one laptop, not to finish a problem set this time, but to build something of their own.

Sheldon: "We've spent a whole semester learning how these models work. Tonight, let's act like a real ML team, pick a problem people actually have, build something that runs, and", he taps the desk, "*prove* it works. That last part is where everyone waves their hands."

Bob: "Prove it works? We just call an LLM. One prompt, done."

Alice (already typing): "Go ahead, ask it something specific it never saw. Last week's lab manual, our department's policy page, your own notes." She turns the screen around. "Confident. And completely wrong."

Sheldon: "That's exactly the point. A single LLM call is often unreliable. A small workflow can fix it, retrieve the facts first, answer from the evidence, then check the answer. But here's the rule: every extra step has to earn its place with evidence."

Bob (frowning): "And how do we know the extra steps actually help? Maybe they just make it slower."

Sheldon (grinning): "Now you're asking the right question, Bob. That question is the project."

Step 0: Find the need

Alice: "People constantly ask questions about a fixed body of text, a product FAQ, a policy document, a set of lecture notes, and they want a correct, grounded answer, not a confident guess."

Sheldon: "Good. Concrete need: *given a small collection of documents, answer questions about them accurately and with evidence, so the user can trust the answer.*"

For this walk-through, suppose the documents are a small online-shop FAQ (returns, shipping, warranty). A user asks: "*How many days do I have to return an item?*" The real answer lives in one FAQ line, "*Returns are accepted within 30 days of delivery.*" --- but the model has never seen our FAQ.

Step 1: The baseline (V0): one LLM call

Bob: "Version 0 is the obvious thing, send the question to the LLM, return whatever comes back."

Alice: "Right, and on questions about *our* private documents it hallucinates, because it has never seen them. This is our baseline. Everything we add has to beat it, or the extra complexity isn't worth it."

```
answer = call_llm(f"Answer the question: {question}")
# call_llm() = one HTTP request to the model's API, a few lines of code
```

Run it on our question and the model *guesses*, it might confidently say "usually 14 days," which is plausible and wrong for our shop. That is the baseline we have to beat.

Step 2: Add retrieval (V1): RAG

Alice: "So before answering, we retrieve the most relevant passages and hand them to the model. The simplest retriever needs no library at all: score each passage by how many words it shares with the question, and take the top ones."

Bob: "Now it answers *from the documents*, not just from memory."

```
# the knowledge base: just a list of passages
passages = [
    "Returns policy: items can be returned within 30 days of delivery.",
```

```

"Shipping: we ship to Hong Kong and Macau; 5 to 7 business days.",
"Warranty: earphones carry a 12-month manufacturer warranty.",
...
]

# retrieval: score by word overlap, take the top k
def retrieve(question, k=2):
    q = set(question.lower().split())
    return sorted(passages, key=lambda p: -len(q & set(p.lower().split())))[:k]

top = retrieve(question)
answer = call_llm(
    "Using ONLY this context, answer the question. "
    "If it is not in the context, say you don't know.\n"
    f"Context: {top}\nQuestion: {question}")

```

Now `top` contains the FAQ line about 30-day returns, so the model answers "30 days", grounded in the document, and you can point to the exact passage it used. *Optional upgrade: replace word overlap with semantic embeddings (e.g. sentence-transformers) for better matching on harder questions; the workflow shape stays exactly the same, and whether the upgrade helps is itself something you can measure.*

Step 3: Add a checking step (V2)

Sheldon: "One more step. After the draft answer, add a second call that checks it against the retrieved context. There are two simple flavours, and these are exactly the other harnesses from the main brief."

(a) Self-check / revise: the model critiques and fixes its own answer:

```

critique = call_llm(
    f"Answer:\n{answer}\n\nContext:\n{context}\n"
    "List any claim in the answer NOT supported by the context; "
    "if all are supported, reply 'OK'.")
if critique.strip() != "OK":
    answer = call_llm(
        f"Rewrite the answer using ONLY supported claims.\n"
        f"Context:\n{context}\nDraft:\n{answer}")

```

(b) A separate verifier (LLM-as-judge): a second call scores or flags the answer:

```

verdict = call_llm(
    f"Context:\n{context}\nAnswer:\n{answer}\n"
    "Is every claim in the answer supported by the context? Reply yes or no.")
# keep the answer if 'yes'; otherwise flag it or fall back to "I don't know"

```

Bob: "So the workflow is retrieve → answer → verify. But we still don't *know* the verifier is useful." **Sheldon:** "Right, that's exactly what Step 4 measures."

Step 4: Prove it

Alice: "So we build an evaluation set, say 50 questions about our documents, each with a reference correct answer and, where possible, the passage in the documents that contains it. Then we decide what 'good' means for this need:"

- **Answer correctness:** does it match the reference answer? (exact match on key facts, human check, or a disclosed LLM-as-judge prompt)
- **Retrieval hit@k:** was the correct source passage among the retrieved chunks?
- **Faithfulness:** is the answer grounded in the retrieved text, or did the model invent claims?

Sheldon: "Run V0, V1, and V2 on the same set, put the numbers in one table, then do the move that turns a demo into science: an ablation. Switch off one component and watch what breaks."

Version	What's in the workflow	Correctness	Faithfulness	Cost / latency
V0 baseline	single LLM call	0.41	0.38	1 call
V1 + retrieval	retrieve → answer	0.76	0.69	2 calls
V2 + verifier	retrieve → answer → verify	0.79	0.91	3 calls
Ablation: V2 without retrieval	answer → verify	0.40	0.39	2 calls

(Illustrative numbers only; yours will differ.) Notice the cells are actual measured values, not words like "better" or "jumps up". Your report must quantify every comparison: give a score, percentage, or count for each version, so the improvement and the ablation are visible as numbers.

Alice: "That's the proof. We didn't just say 'RAG is better', we showed with numbers what improves, what fails, and which component earns its place."

Sheldon: "And notice, the evaluation design was the real work. Correctness, faithfulness, hit@k fit *this* QA need. Your own need may demand entirely different metrics, that's your job to figure out."

The transferable lesson

1. State the need and define what "good" means for it.
2. Build a baseline, usually a single LLM call.
3. Build a small improved workflow (harness) by composing one or more simple techniques.
4. Design an evaluation with ground truth or verifiable criteria, and metrics that fit your need.
5. Prove improvement over the baseline, and run an ablation/control to show which component matters.

Appendix C: Suggested References

Optional starting points for reading more about LLM evaluation and related methods:

1. Laskar, Md Tahmid Rahman, et al. "A systematic study and comprehensive evaluation of ChatGPT on benchmark datasets." arXiv:2305.18486 (2023).
2. He, Chaoqun, et al. "UltraEval: A Lightweight Platform for Flexible and Comprehensive Evaluation for LLMs." arXiv:2404.07584 (2024).
3. Peng, Ji-Lun, et al. "A Survey of Useful LLM Evaluation." arXiv:2406.00936 (2024).
4. Chang, Yupeng, et al. "A survey on evaluation of large language models." ACM TIST 15.3 (2024): 1-45.

5. Fan, Lizhou, et al. "NPHardEval: Dynamic benchmark on reasoning ability of large language models via complexity classes." arXiv:2312.14890 (2023).